Design Document

for

UniPool

Version 1.0

Prepared by

Group #16 **Group Name: Undefined_Behaviour**

Aalok	240011	aalok24@iitk.ac.in
Alankrit Gupta	220110	alankritg22@iitk.ac.in
Anubhav Rana	210162	anubhavr21@iitk.ac.in
Aryan Nigam	220227	naryan22@iitk.ac.in
Bhavishya Gupta	220295	bhavishyap22@iitk.ac.in
Gagandeep Kolambe	220393	gagandeep22@iitk.ac.in
Harshit Gupta	220439	gharshit22@iitk.ac.in
Karan Kumar	230533	karanm23@iitk.ac.in
Naman Mohan Singh	230678	mohans23@iitk.ac.in
Pavi Agarwal	220762	apavi22@iitk.ac.in

Course: CS253

Mentor TA: V Arvind

Date: 3 February 2026

CONTENTS II

REVISIONS III

1 CONTEXT DESIGN **1**

1.1 CONTEXT MODEL 1

1.2 HUMAN INTERFACE DESIGN 3

2 ARCHITECTURE DESIGN 12

3 OBJECT ORIENTED DESIGN **1**4

3.1 USE CASE DIAGRAM 14

3.2 CLASS DIAGRAM 17

3.3 SEQUENCE DIAGRAMS 19

3.4 STATE DIAGRAM 29

4 PROJECT PLAN **32**

4.1 TIMELINE 32

4.2 DEVELOPMENT PHASES 33

APPENDIX A – GROUP LOG **35**

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Aalok	Preliminary draft	03/02/26
	Alankrit Gupta		
	Anubhav Rana		
	Aryan Nigam		
	Bhavishya Gupta		
	Gagandeep Kolambe		
	Harshit Gupta		
	Karan Kumar		
	Naman Mohan Singh		
	Pavi Agarwal		

1 Context Design

1.1 Context Model

The UniPool system is designed as an integrated set of interdependent subsystems that collectively enable secure and efficient campus carpooling. These subsystems are tightly coupled with the Firebase backend to ensure persistent data storage and real-time synchronization across user sessions. The architecture supports seamless transitions between functional modules, such that actions in one subsystem, such as ride selection, automatically trigger related processes, such as chat initiation.

The major subsystems of UniPool include:

1. **User Authentication and Security System:** Manages user registration and secure login, ensuring that only authenticated students can access the platform using OTP.

2. **User Profile and Trust Management System:** Handles profile creation, display names, profile pictures, and maintains the system-generated "Rides Completed" trust metric.

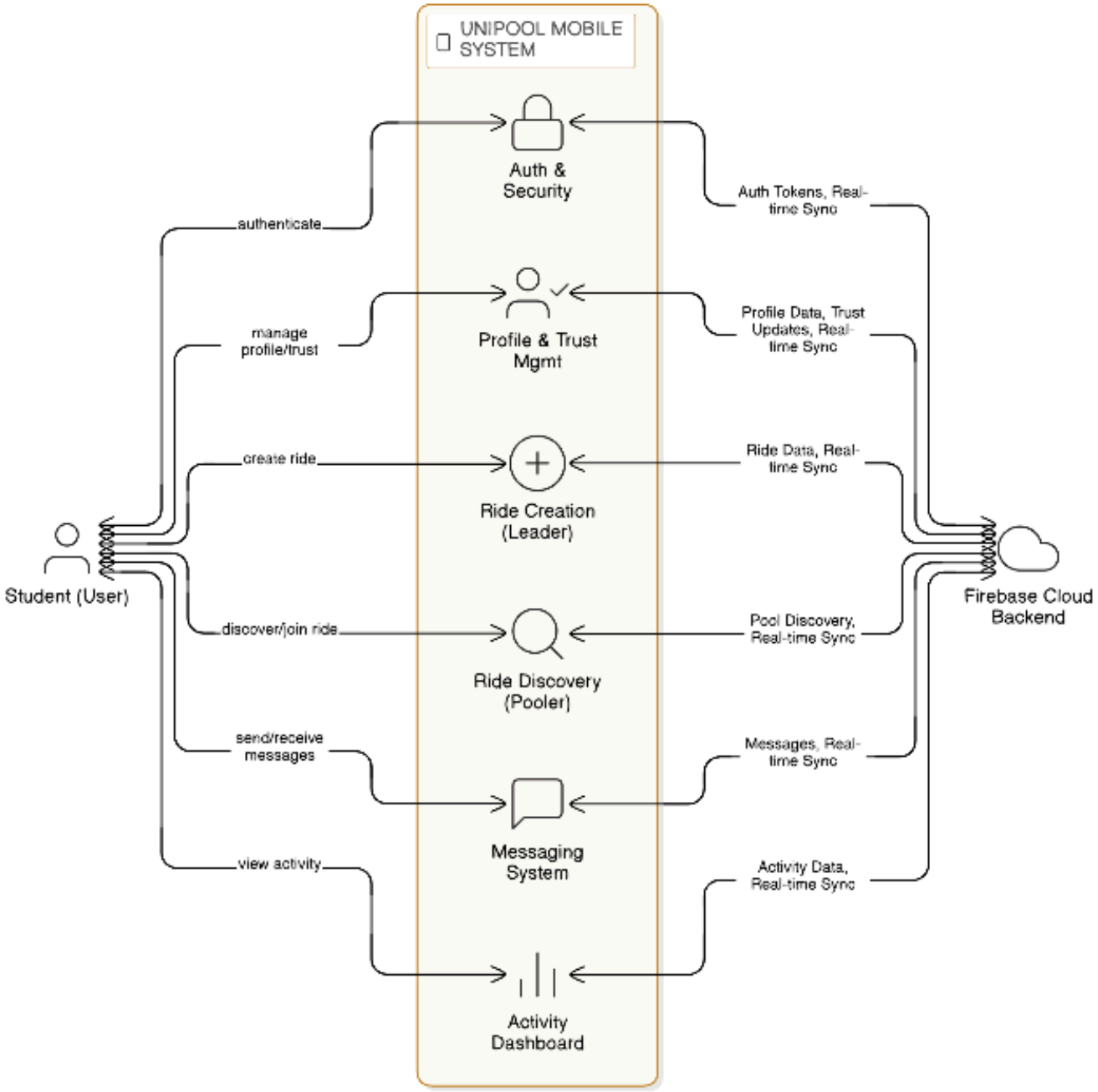
3. **Ride Creation and Leadership System:** Enables Leaders to post rides with source, destination, date, and time details, making them visible in the global ride feed.

4. **Ride Discovery and Filtering System:** Allows Poolers to browse, search, and filter available rides efficiently based on travel preferences.

5. **Real-Time Messaging and Coordination System:** Provides in-app chat for ride coordination without sharing personal contact information.

6. **Ride Management and Activity Dashboard:** Tracks active rides for both Leaders and Poolers and provides a unified view of ongoing engagements.

7. **Trust Verification System:** Allows Poolers to preview a Leader's trust profile before committing to a ride.



1.2 Human Interface Design

1.2.1 User Flow Description

The human interface of UniPool follows a mobile-first design philosophy, emphasizing simplicity, clarity, and rapid task completion for students coordinating rides on the go. The interface uses a clean visual theme and intuitive navigation.

The primary interface components and user interaction flows are as follows:

Authentication and Onboarding Interface:
Users access the system through a minimalist login and registration screen with real-time validation for email and password inputs.

Command Dashboard (Home Screen):
The home screen serves as the central navigation hub, offering two primary actions: "Be a Leader" and "Be a Pooler," along with quick access to Profile and My Activity.

Ride Specification Interface (Leader Flow):
Leaders enter ride details through a structured form with native date and time pickers to reduce input errors.

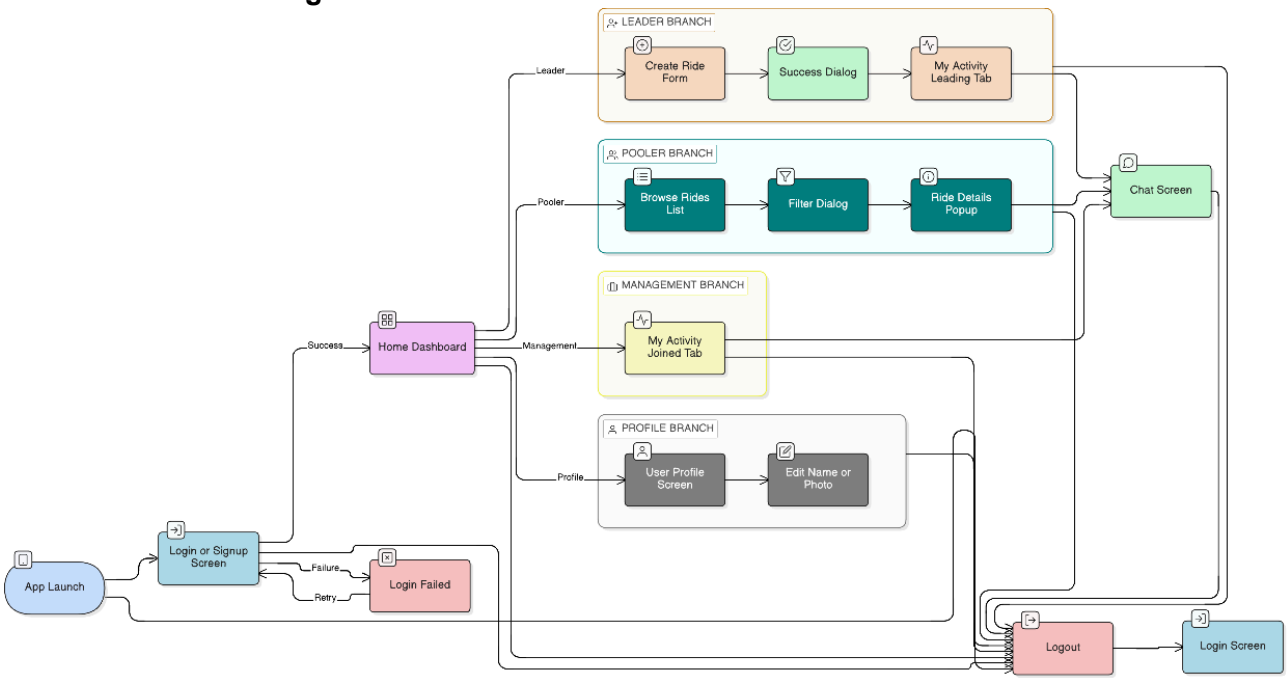
Marketplace Feed (Pooler Flow):
Poolers browse available rides in a scrollable list of ride cards with filtering options and drill-down access to ride details and trust information.

Synchronous Chat Interface:
The chat screen follows standard mobile messaging conventions with message bubbles and timestamps for coordination.

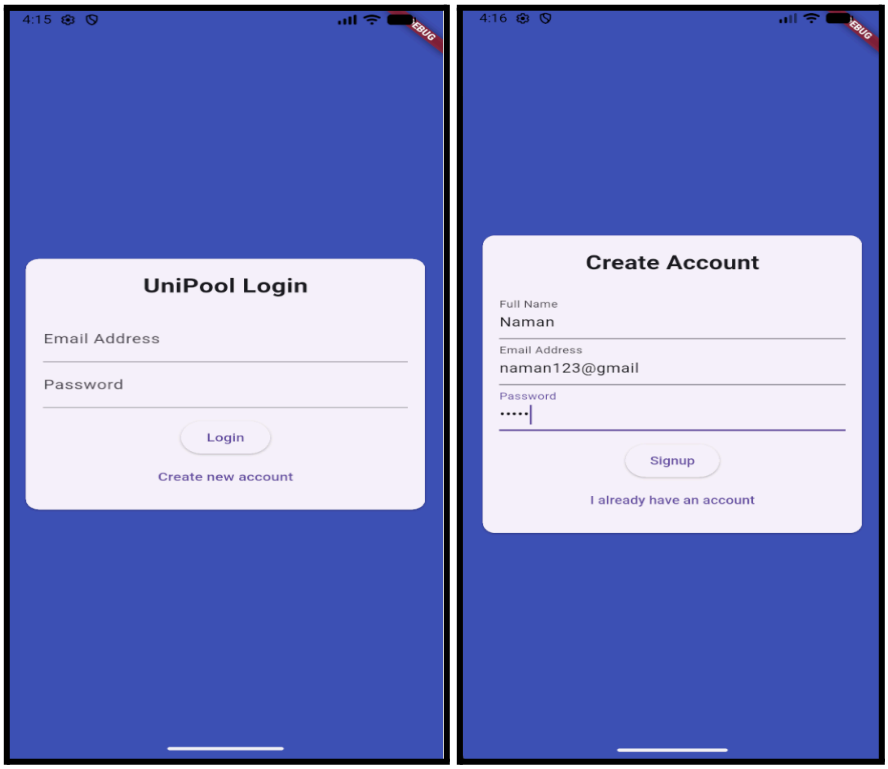
Trust-Centric Profile View:
User profiles display public identity and the Rides Completed trust metric. Profiles are read-only for other users and editable for the account owner.

Unified Activity Hub:
The My Activity screen provides a tab-based view of rides the user is leading and rides the user has joined.

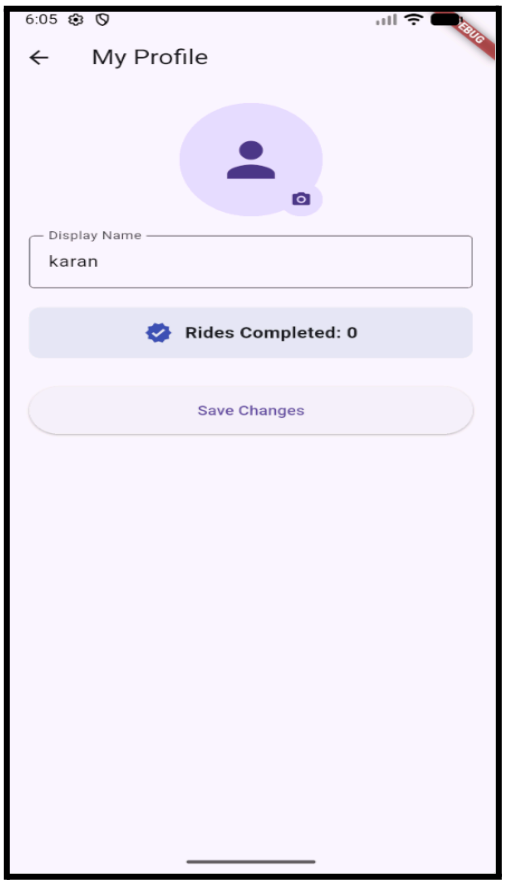
1.2.2 User Flow Design



1.2.2.1 Sign Up and Login Page

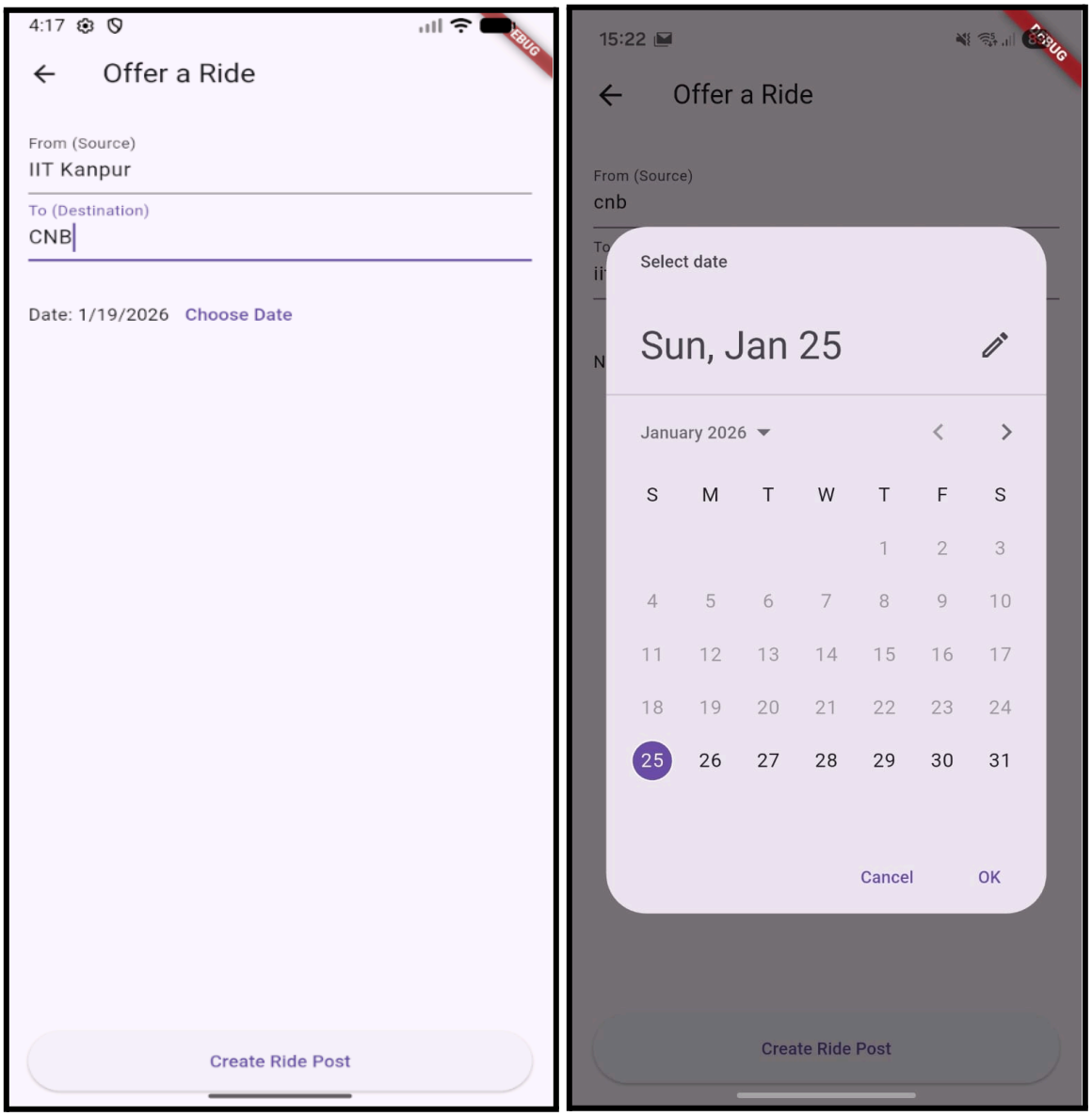


1.2.2.2 Profile Creation Page

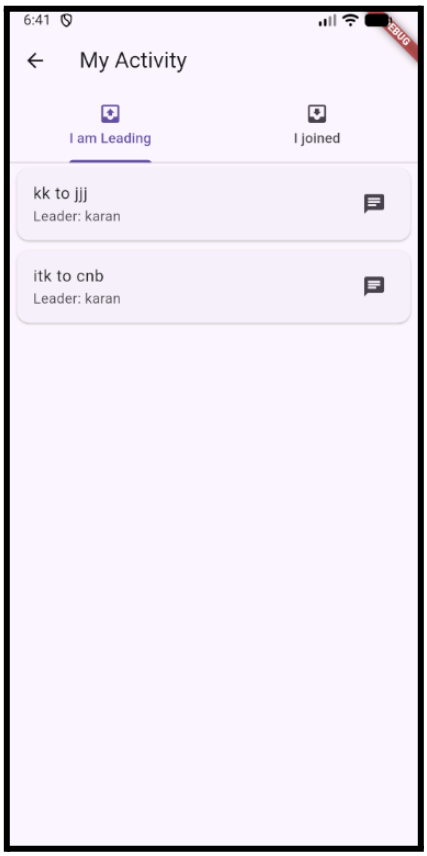


1.2.2.3 Homepage

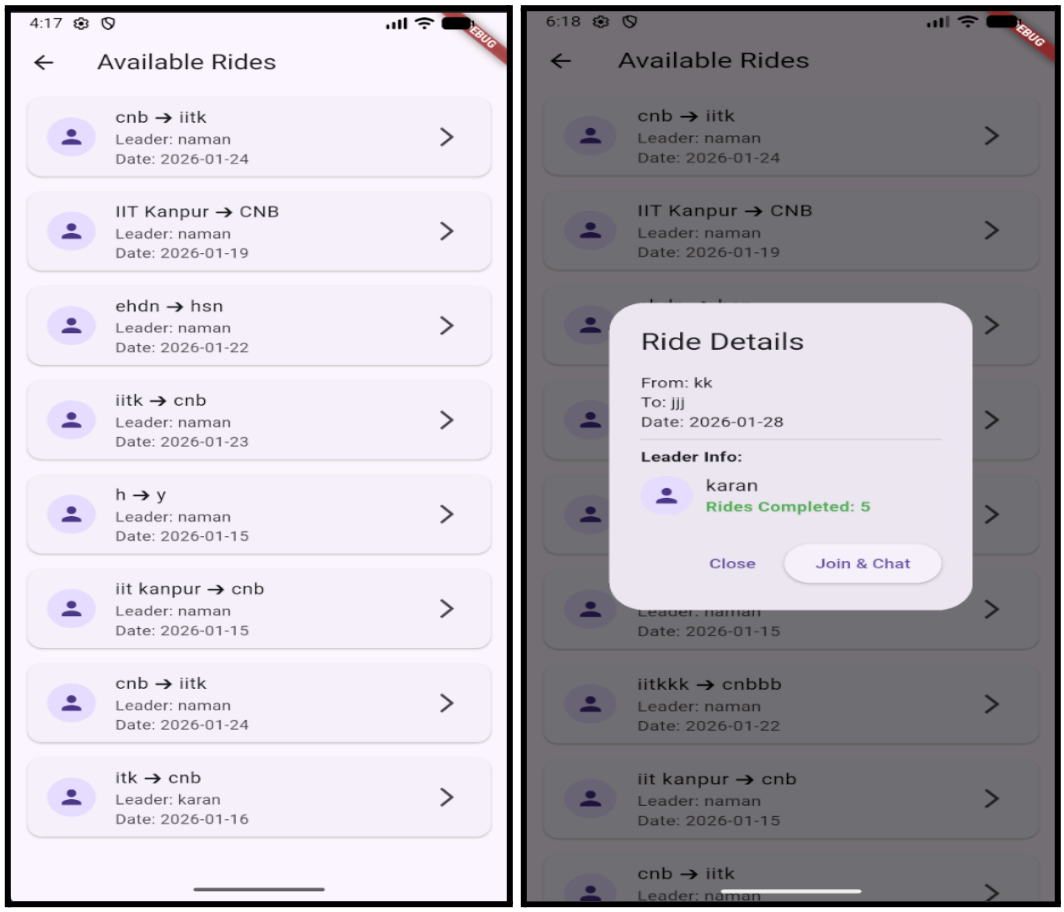
1.2.2.4 Create Ride Screen



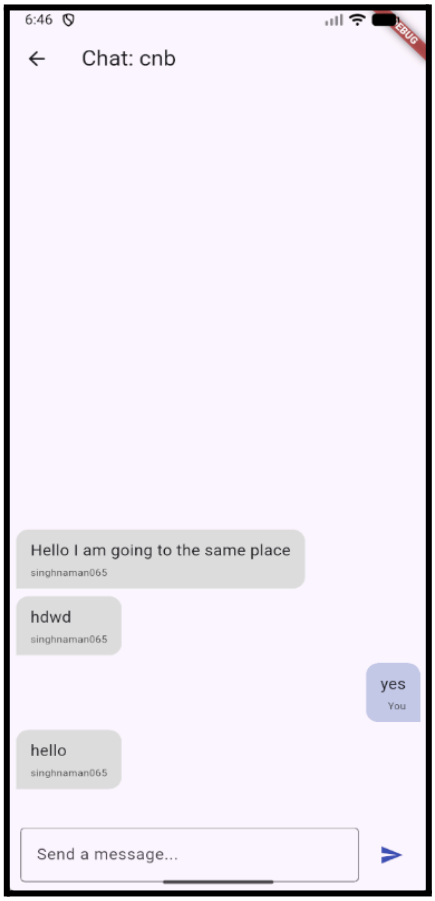
1.2.2.5 My Activity Screen

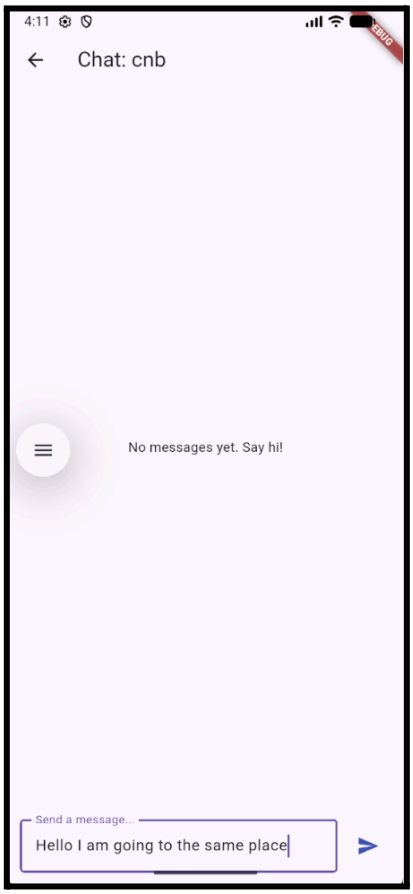


1.2.2.6 Browse Rides Screen and View Profile



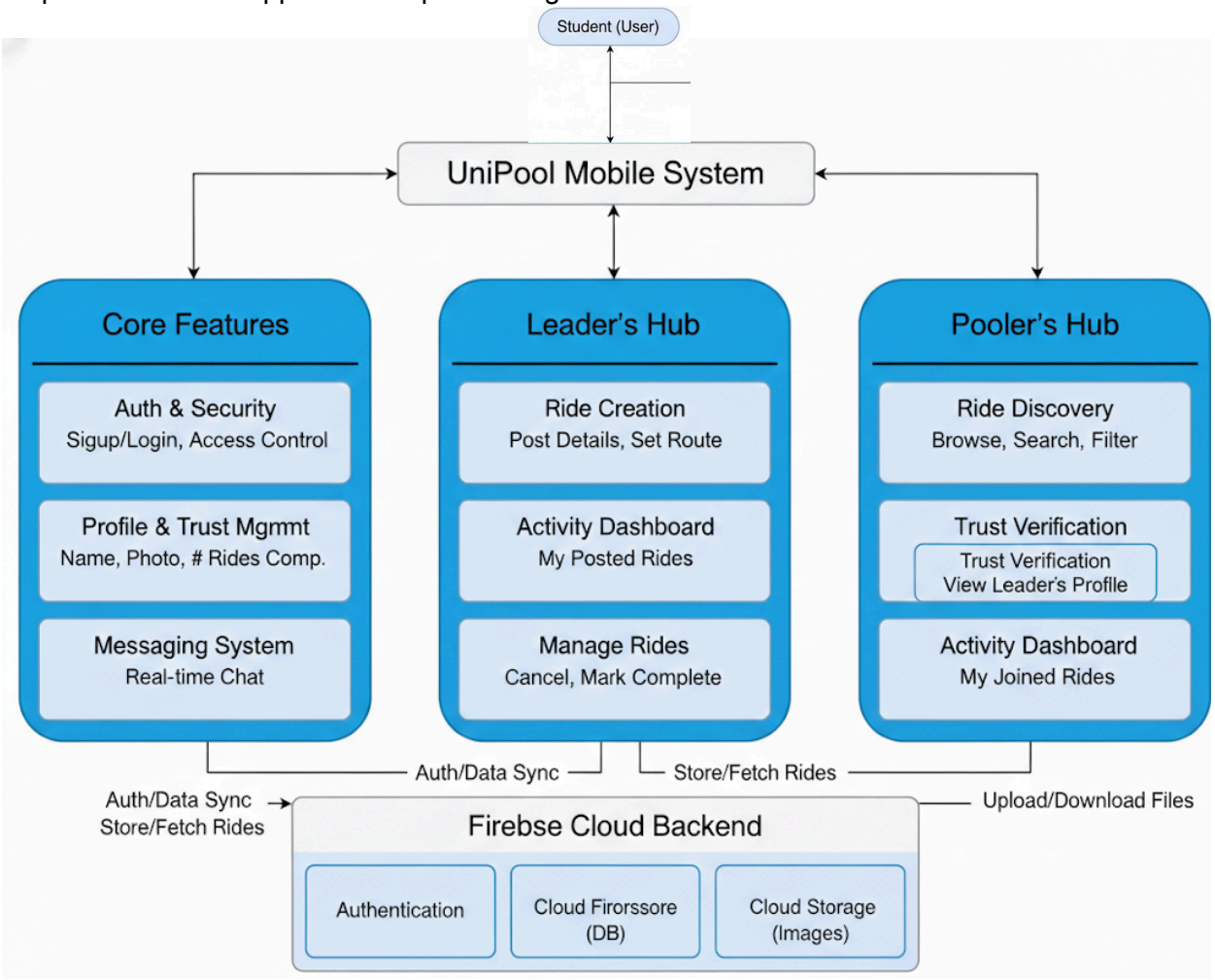
1.2.2.7 Chat Interface Screen





2 Architecture Design

Unipool is a mobile application implementing a client-server architecture.



The client-server architecture enables decentralised access from client devices to a centralised, shared database from multiple locations. The setup ensures that all users can access the same information since only data stored at the server needs to be kept up to date. This allows seamless searches, reporting and claim management across the platform.

Significant advantages of the client-server model include:

- **Centralised services:** Core functionality such as item posting and user authentication are centralised, mitigating the need for redundant implementations on individual client devices.
- **Real-time updates:** Since operations interact with a shared database, updates (such as new posts) are immediately reflected for all users.

Some downsides of the client-server model include:

- **Single point of failure:** If the central server goes offline, some of the core functionality of the application becomes unavailable.
- **Network dependency:** The application requires a stable internet connection to function effectively, which may prove a limitation in certain areas.

The model meets the security requirements as database access requires authentication and physical access to the server is not possible as the backend is hosted on the cloud.

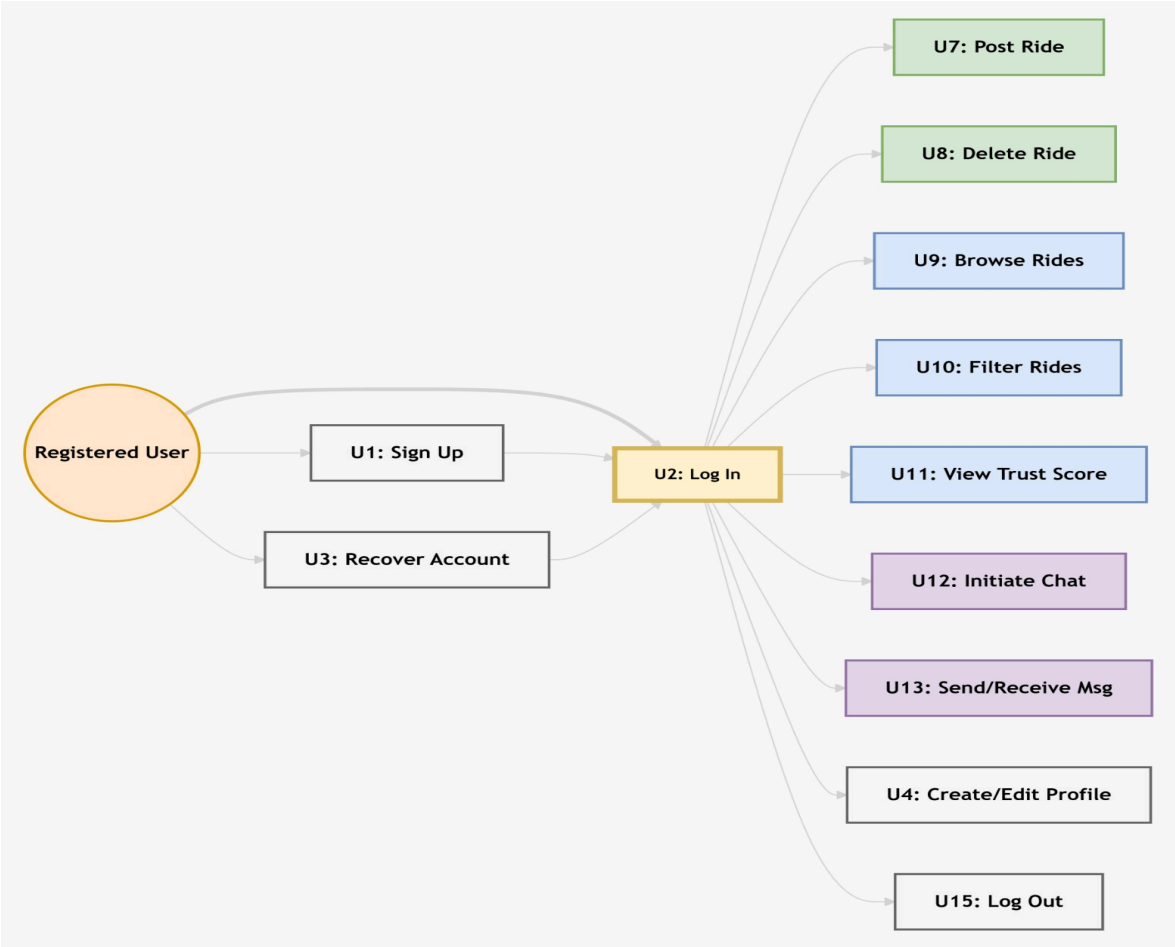
However, performance requirements are limited by the performance and capacity of the server, with possible latency in response under high load. Furthermore, changes to the backend may necessitate the deployment of updates to the frontend as well, complicating maintenance and version compatibility.

At a high level, the system is divided into two primary tiers:

1. **Presentation Tier (Mobile Client):** Developed using the Flutter framework, this tier manages the state of the application and handles user interactions. It uses a reactive programming model to update the UI instantly when data changes in the backend.
2. **Data & Logic Tier (Firebase Cloud):** Instead of a traditional monolithic server, UniPool utilizes Firebase's decentralized services:
    a. **Firebase Authentication:** Handles secure session management and identity verification.
    b. **Cloud Firestore:** A NoSQL document database that stores ride details, user profiles, and chat messages. It provides real-time listeners that "push" updates to the mobile client without the need for manual polling.
    c. **Firebase Storage (Optional):** Used for hosting user-uploaded profile images.

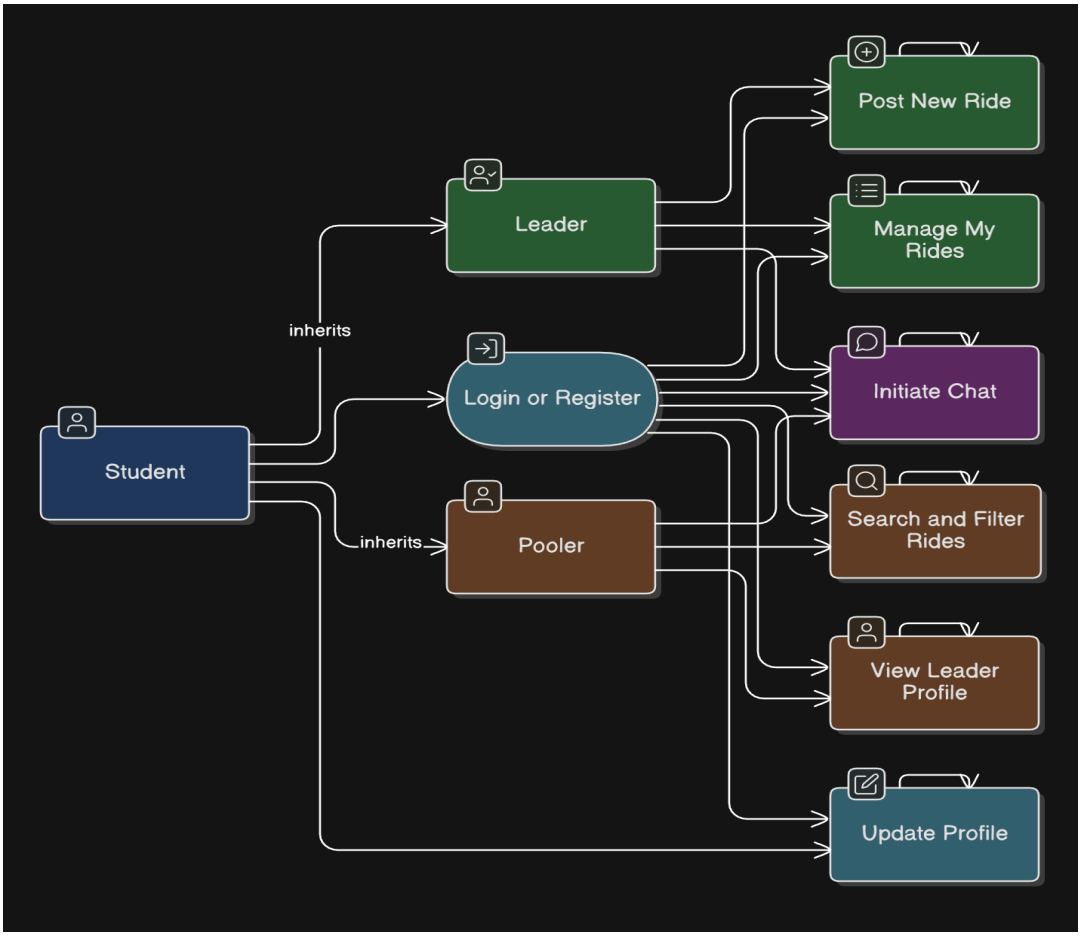
3 Object Oriented Design

3.1 Use Case Diagram

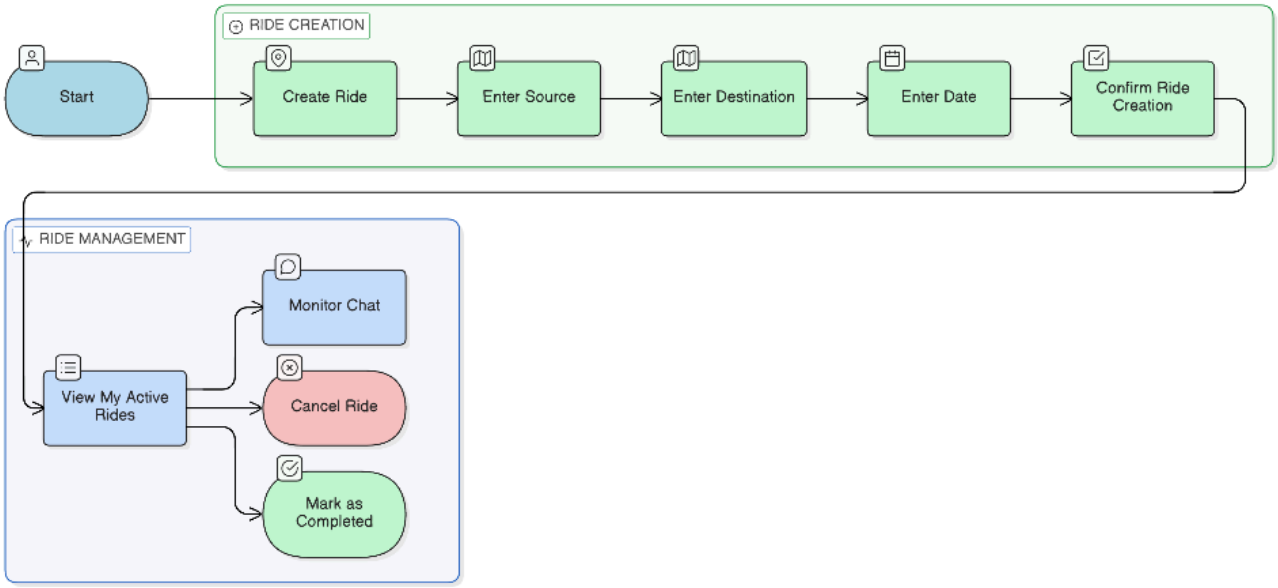


Registered users shall fall into two roles: Pooler and Leader. Both can:

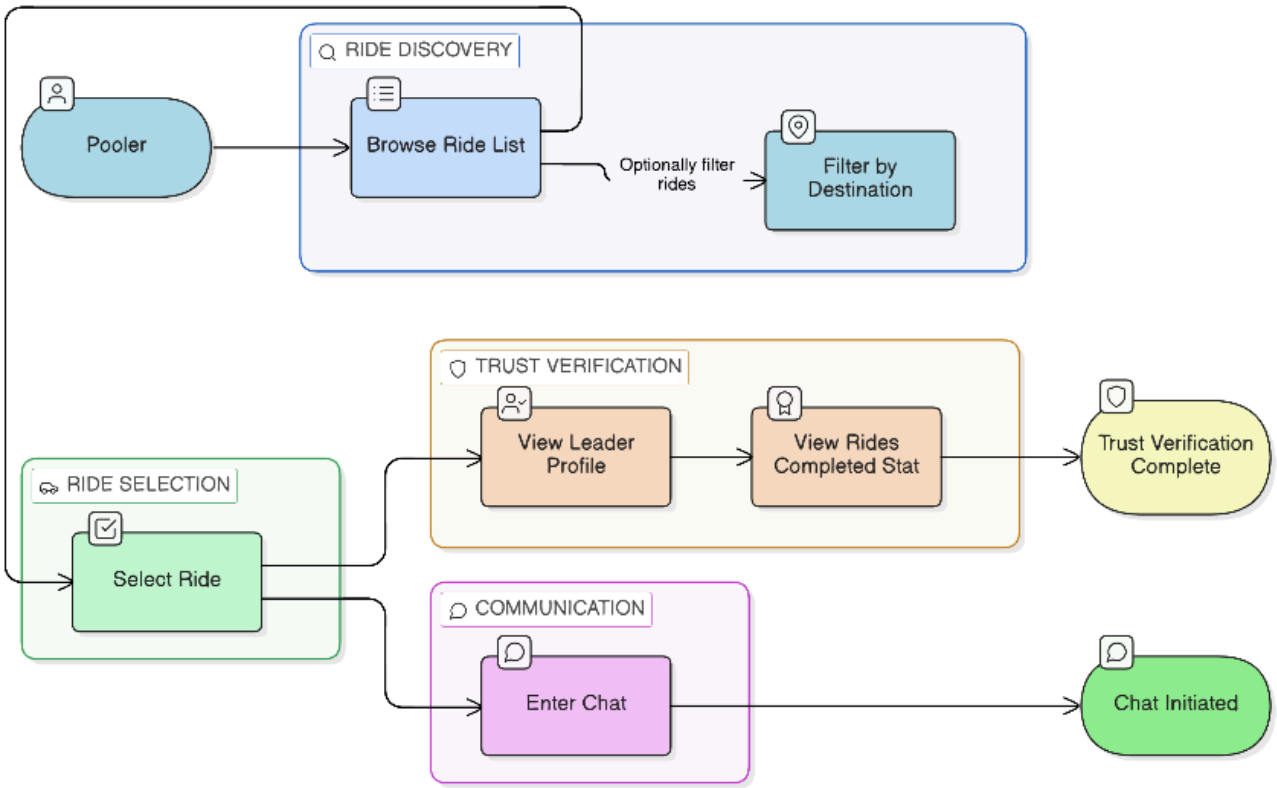
- Sign up and log in.
- Create Ride (Leader) and Browse Rides (Pooler).
- Initiate chat for Ride
- View Profile
- Create-Edit Profile.



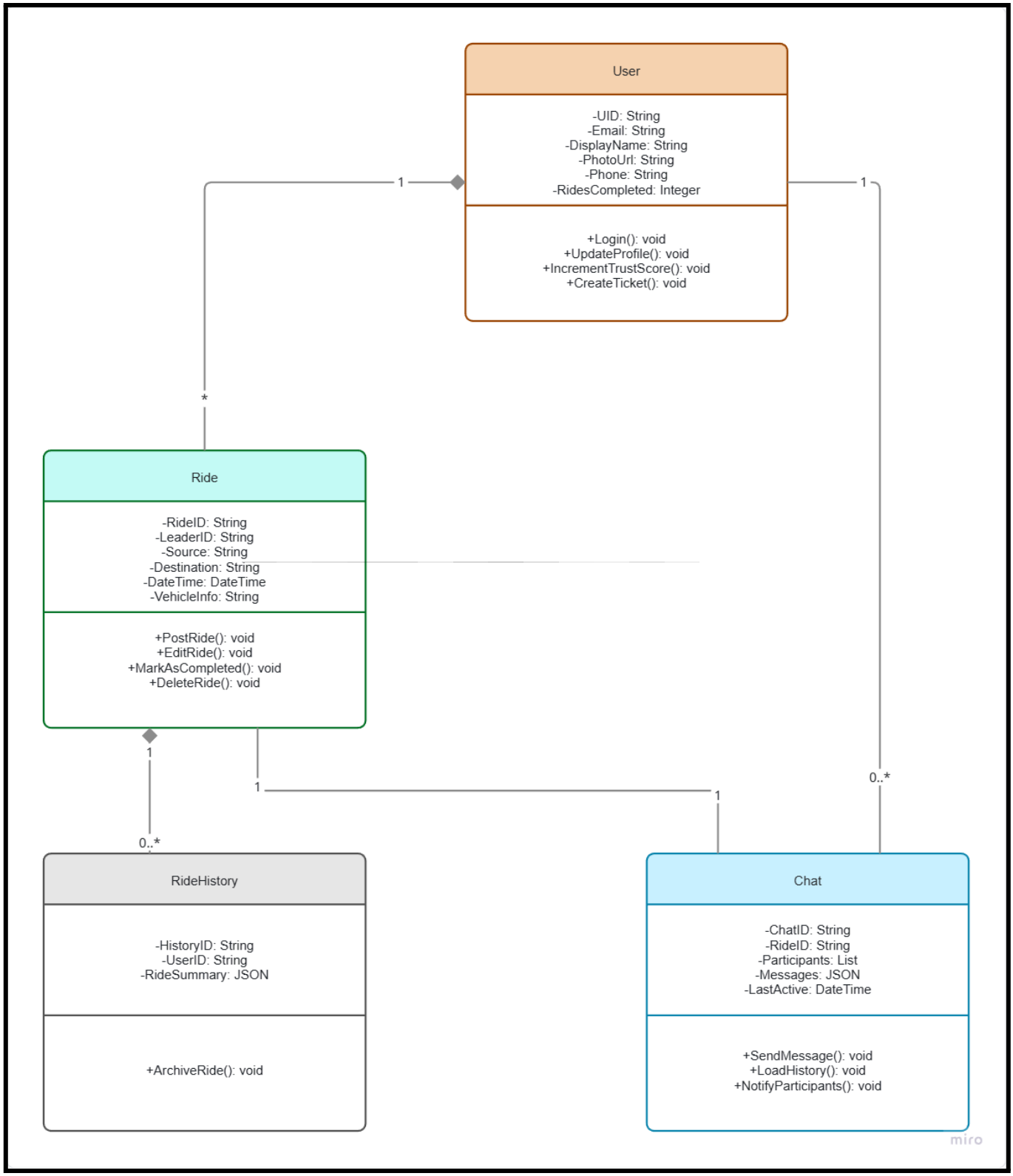
Use Case: Post Ride (Leader)



Use Case: Browse Available Rides (Pooler)



3.2 Class Diagram



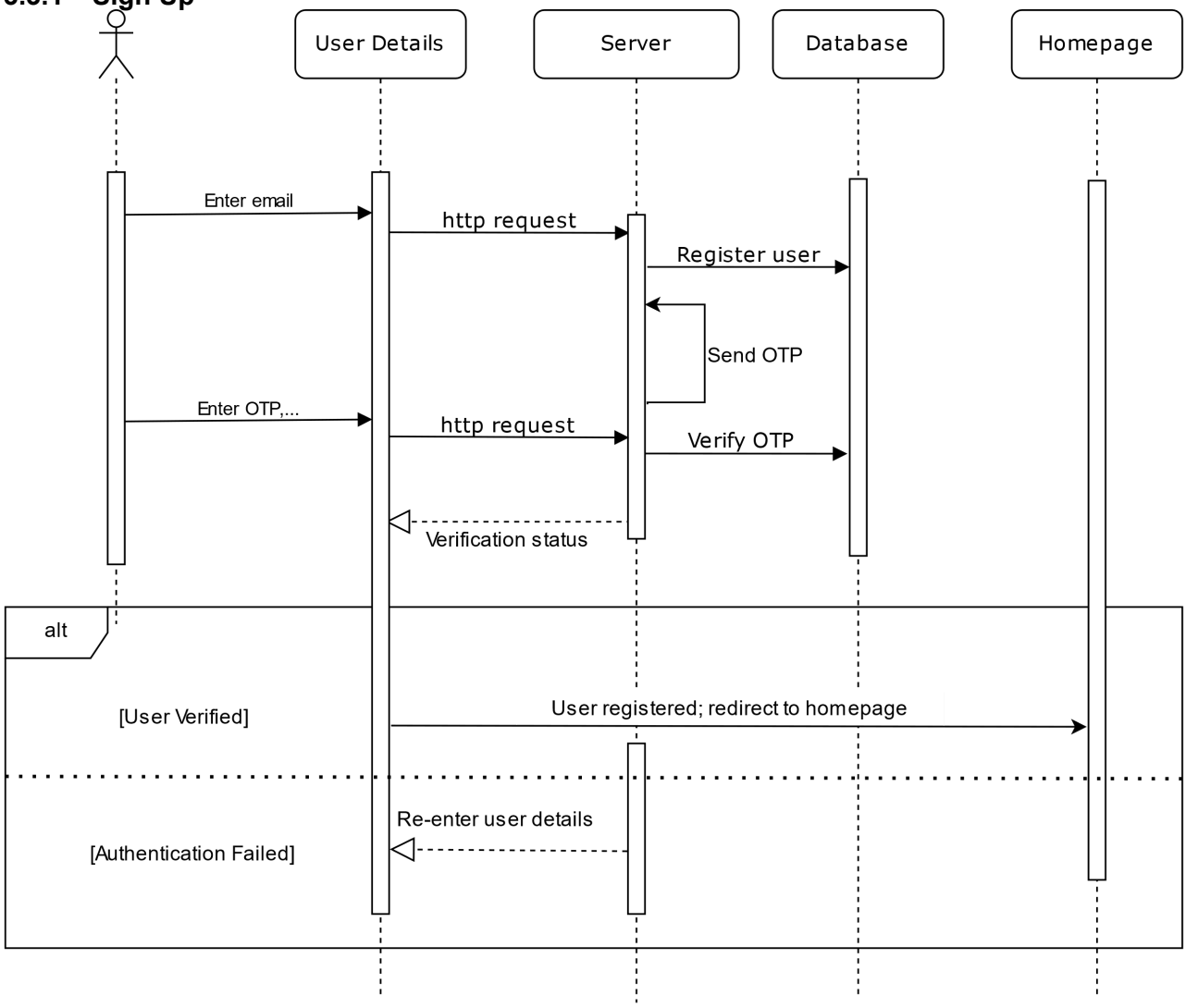
The UniPool application follows a robust object-oriented design paradigm to ensure modularity and scalability. The system is modeled using the following primary classes:

- **User:** Represents the base entity for all authenticated students. It encapsulates personal details and trust metrics (e.g., `ridesCompleted`).
- **Ride:** A core entity representing a carpooling event. It contains travel details and links the Leader to potential Poolers.
- **Chat:** Manages the communication context for a specific ride.
- **Message:** Represents individual text exchanges within a Chat.

Data members of each class are encapsulated (private) and accessed via getters and setters, ensuring data integrity. Public methods are responsible for triggering state changes, such as posting a ride or sending a message, which then synchronize with the Firebase backend.

3.3 Sequence Diagrams

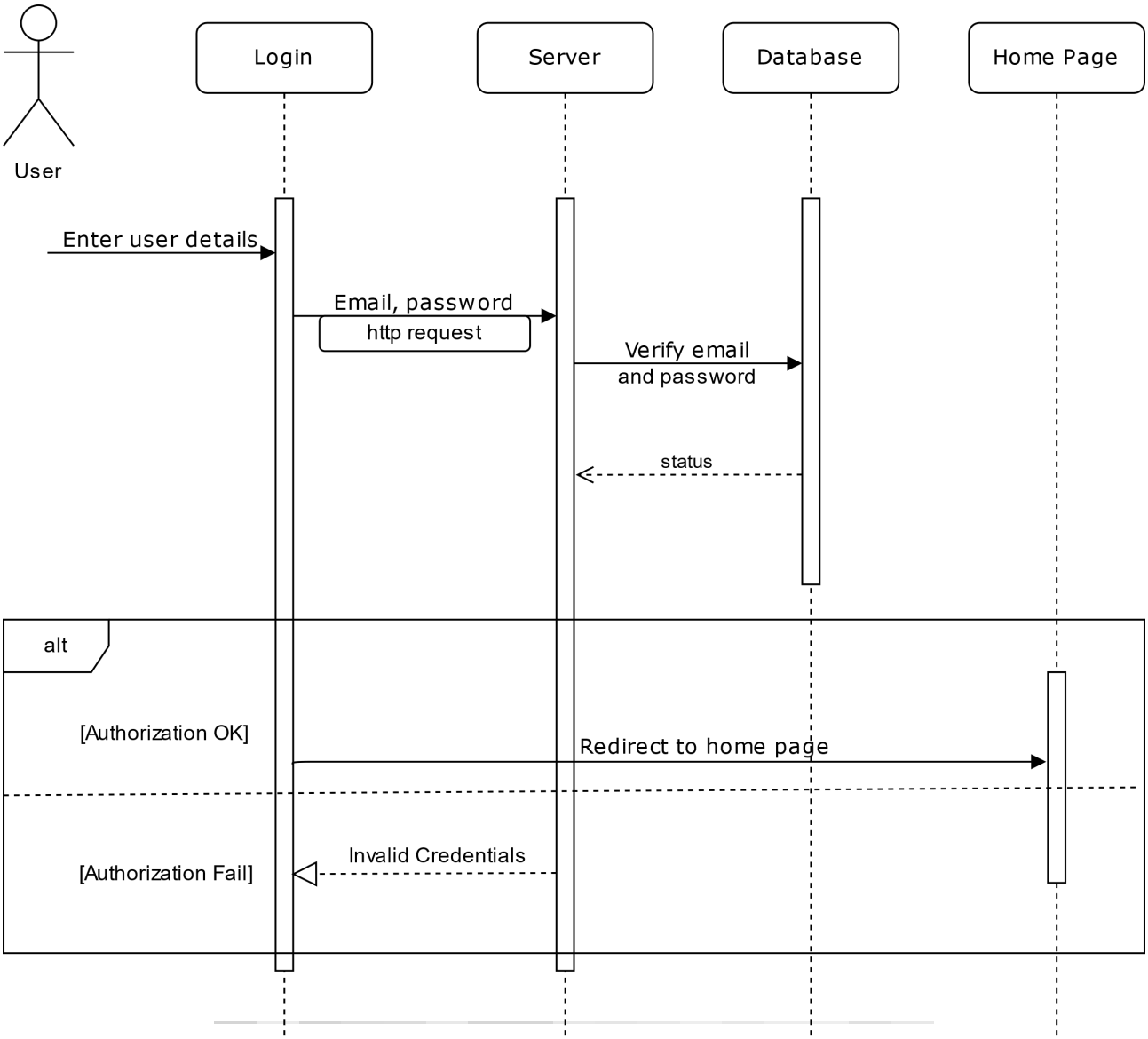
3.3.1 Sign Up



Purpose:
The signup page is designed to facilitate seamless access to UniPool for users affiliated with IIT Kanpur. It ensures that only authorised individuals can create/browse rides within the IITK community.

User authentication and role-based access control ensure that only authorized users can access the system and perform permitted actions such as creating, joining, or managing rides. Institute-based verification restricts platform usage to IITK-affiliated users, thereby enhancing trust, security, and reliability within the UniPool community.

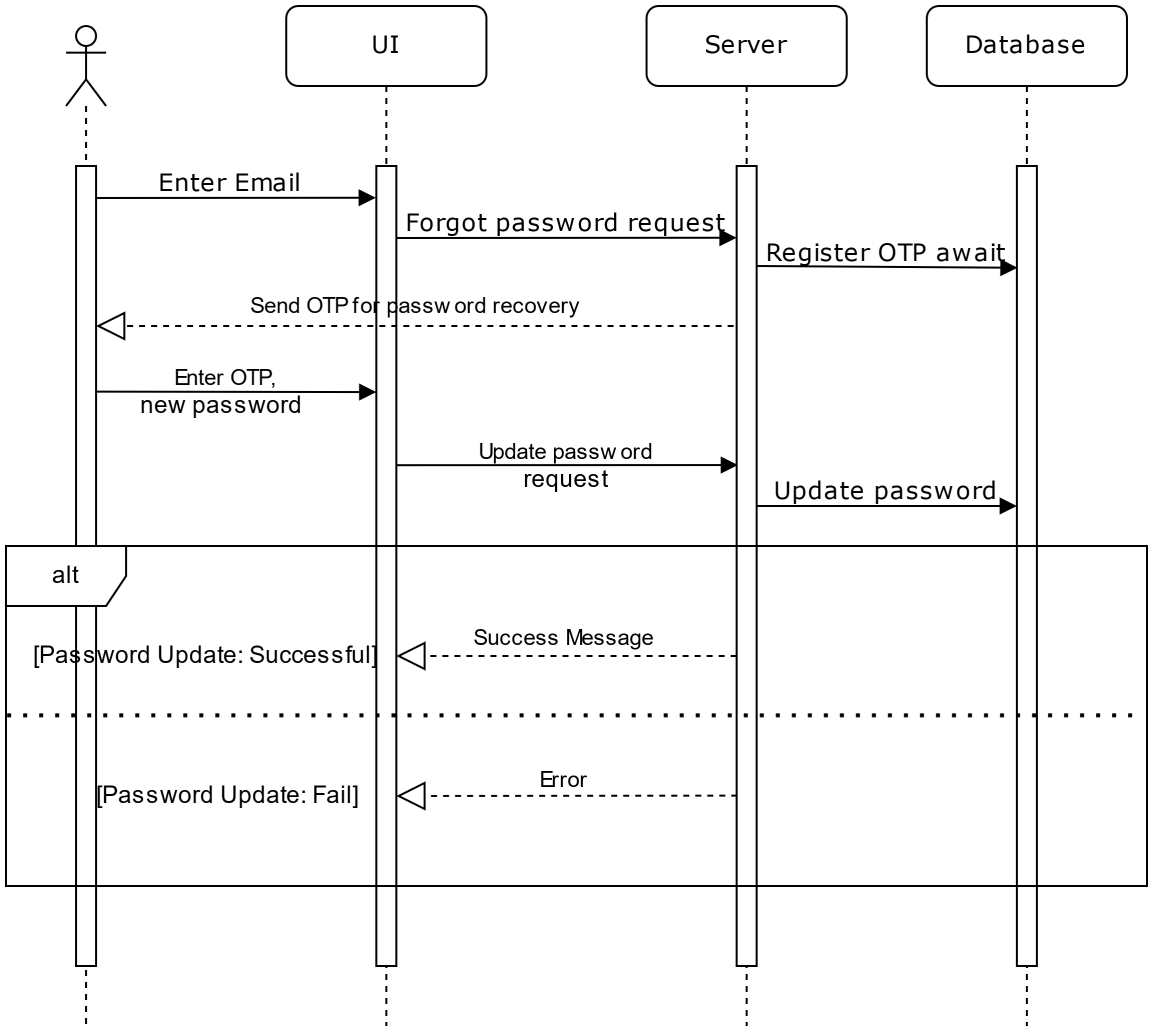
3.3.2 Login



Purpose:
The login page serves as the gateway for authenticated users to access UniPool, ensuring secure entry into the system.

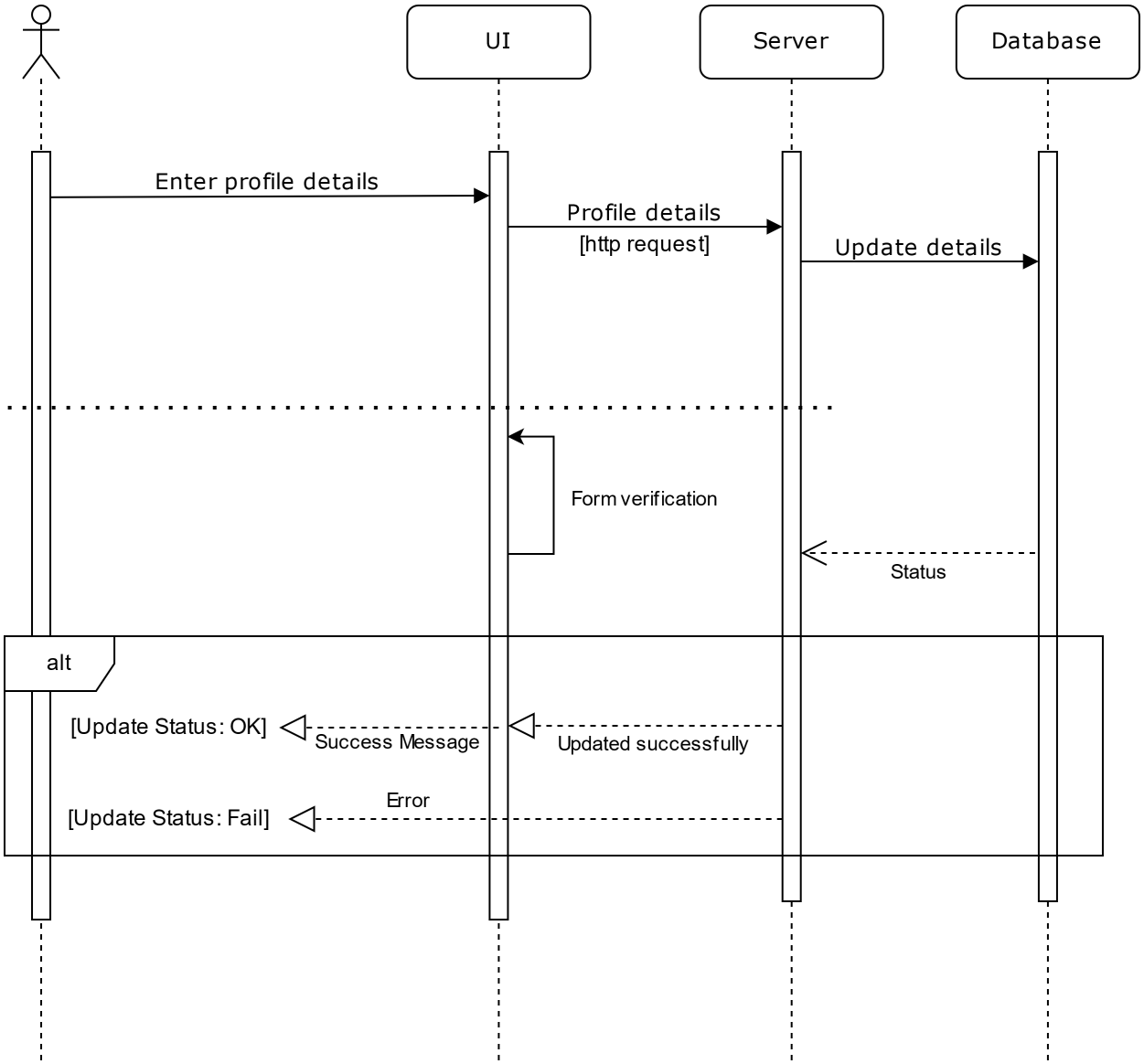
- **Secure authentication:** Restricts access to registered IITK users.
- **Session management**: Allows users to continue activities and track queries.
- **Efficient navigation**: Directs users to HomePage.

3.3.3 Reset Password / Recover Account



Purpose:
This use case ensures that users can change their password to securely reestablish access to their account.

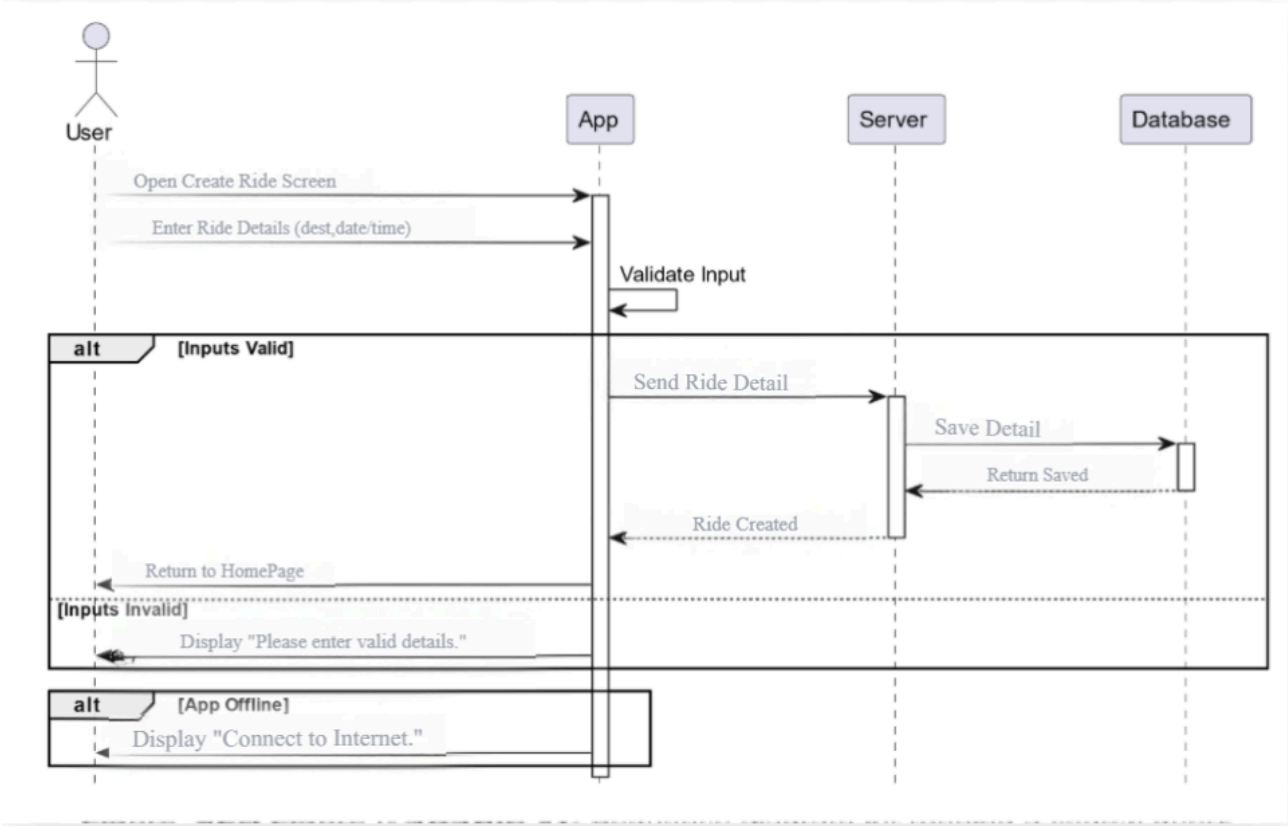
3.3.4 Update/Create Profile



Purpose:
The 'Update Profile' feature allows IITK users to modify their personal details, ensuring accurate and up-to-date information. It helps in better identification, communication within UniPool.

- **Profile Accuracy**: Enables users to keep their information up to date.
- **Better Identification**: Helps in building trust and make informed decisions.
- **Enhanced Communication**: Ensures correct contact details for rides.

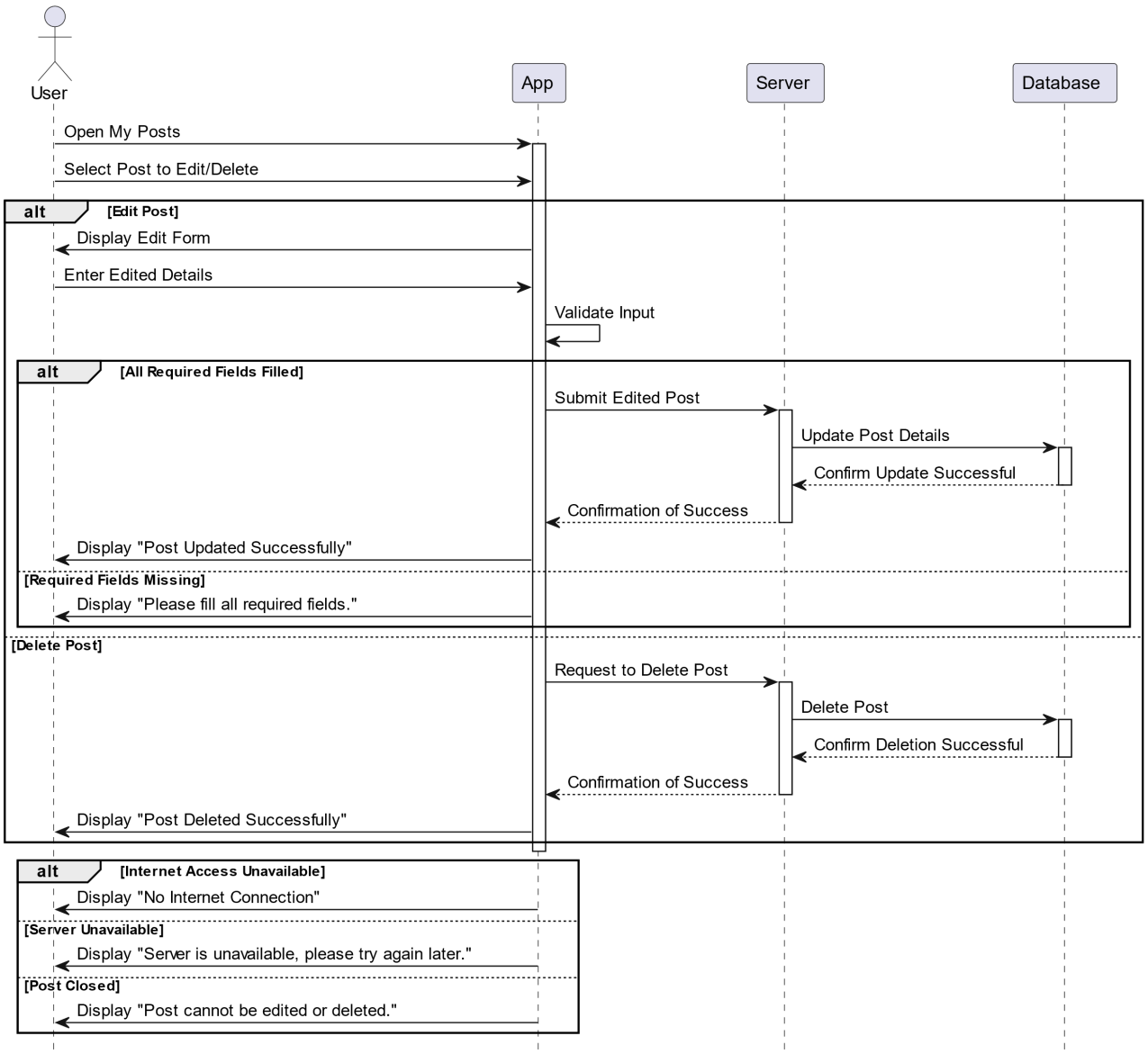
3.3.5 Post a Ride (Leader Flow)



Purpose: The ‘Post a Ride’ feature allows students to broadcast their travel plans to the campus community. It automates the entry of trip logistics into the central database for discovery.

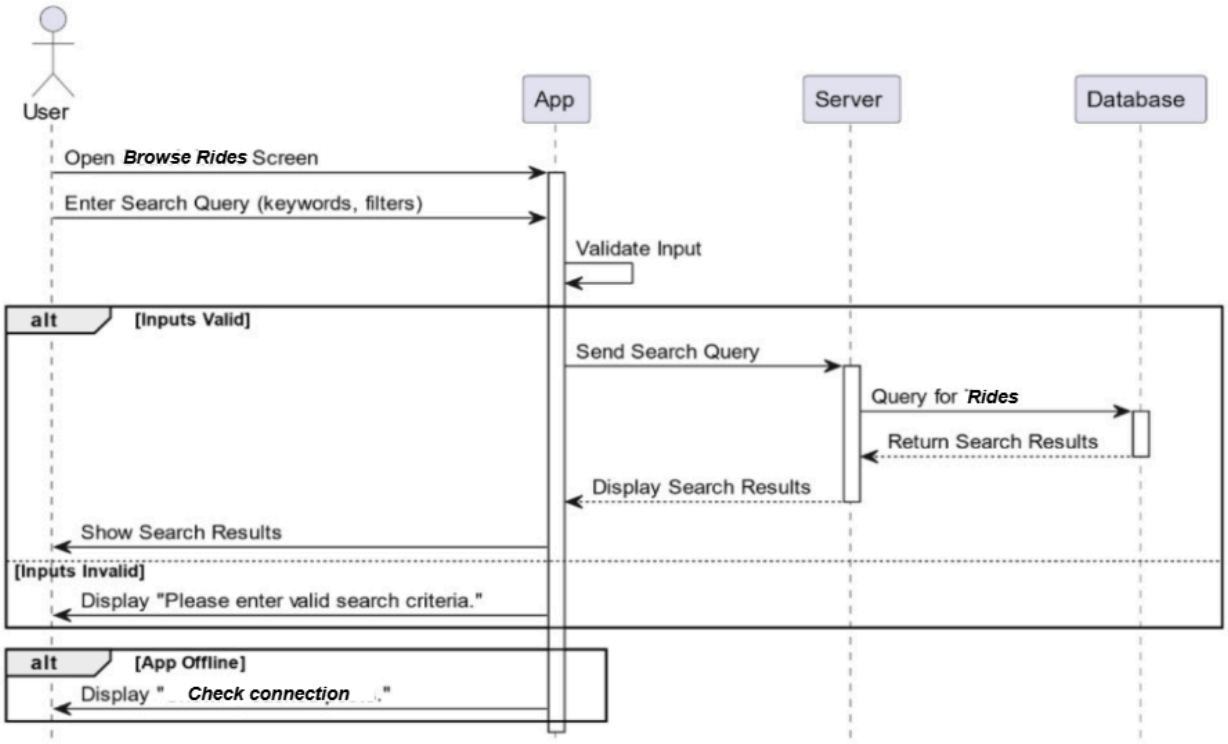
- **Trip Publication:** Enables Leaders to list origin, destination, and timing.
- **Data Integrity:** Validates that ride times are set in the future.
- **Resource Sharing:** Facilitates the efficient use of available vehicle seats.

3.3.7 Create/Edit/Delete Ride Posts



Purpose:
If a user has made some error or wants to change some details of his/her ride already posted then s/he can edit the details via the edit/delete feature provided in `My Activity` → `Leading` → `Select the ride` → `Edit/Delete`.

3.3.8 Browse & Filter Rides (Pooler Flow)

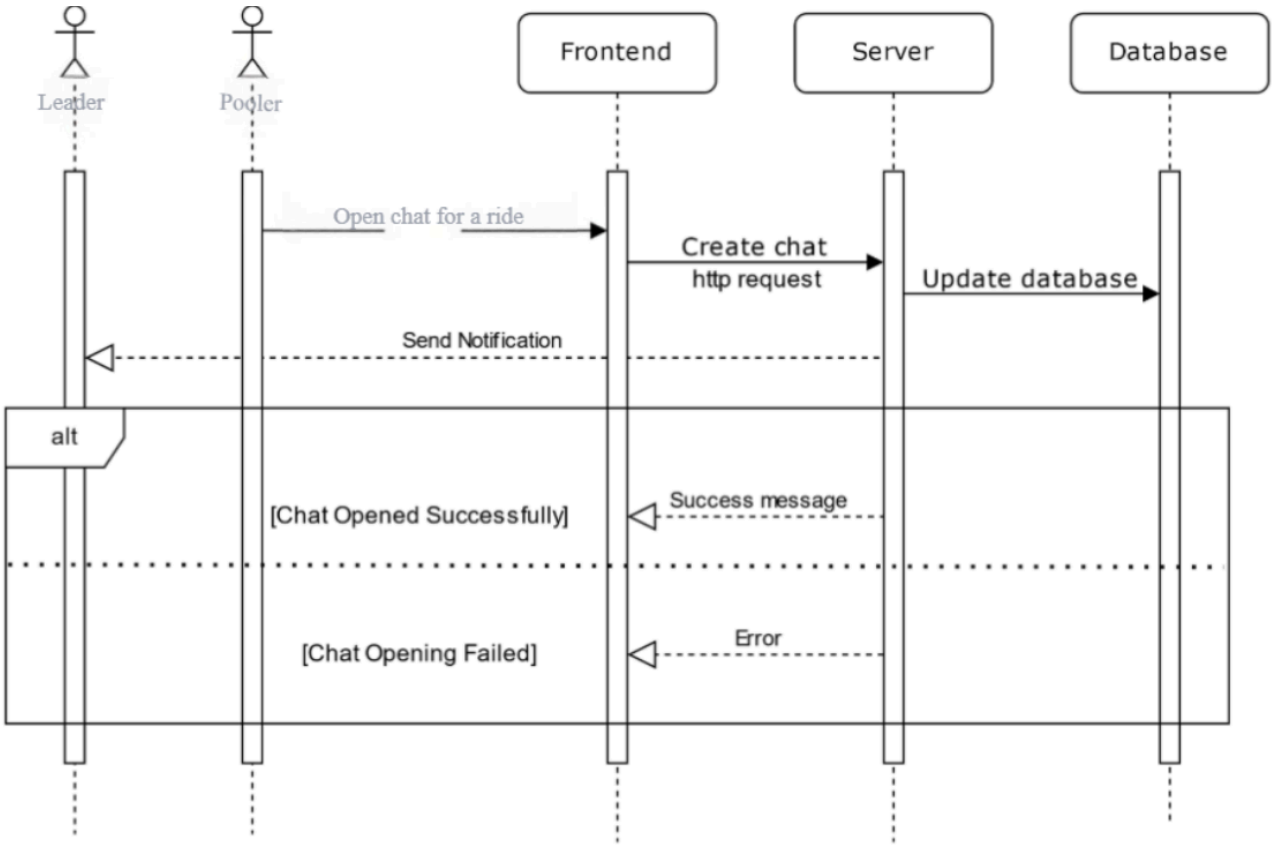


Purpose:

This feature enables Poolers to quickly locate rides that match their specific travel needs through real-time database queries.

1. **Dynamic Discovery:** Provides a filtered view of available carpools.
2. **Efficient Navigation:** Sorts results to show the most relevant upcoming trips.
3. **Time Saving:** Reduces the effort needed to find compatible travel partners.

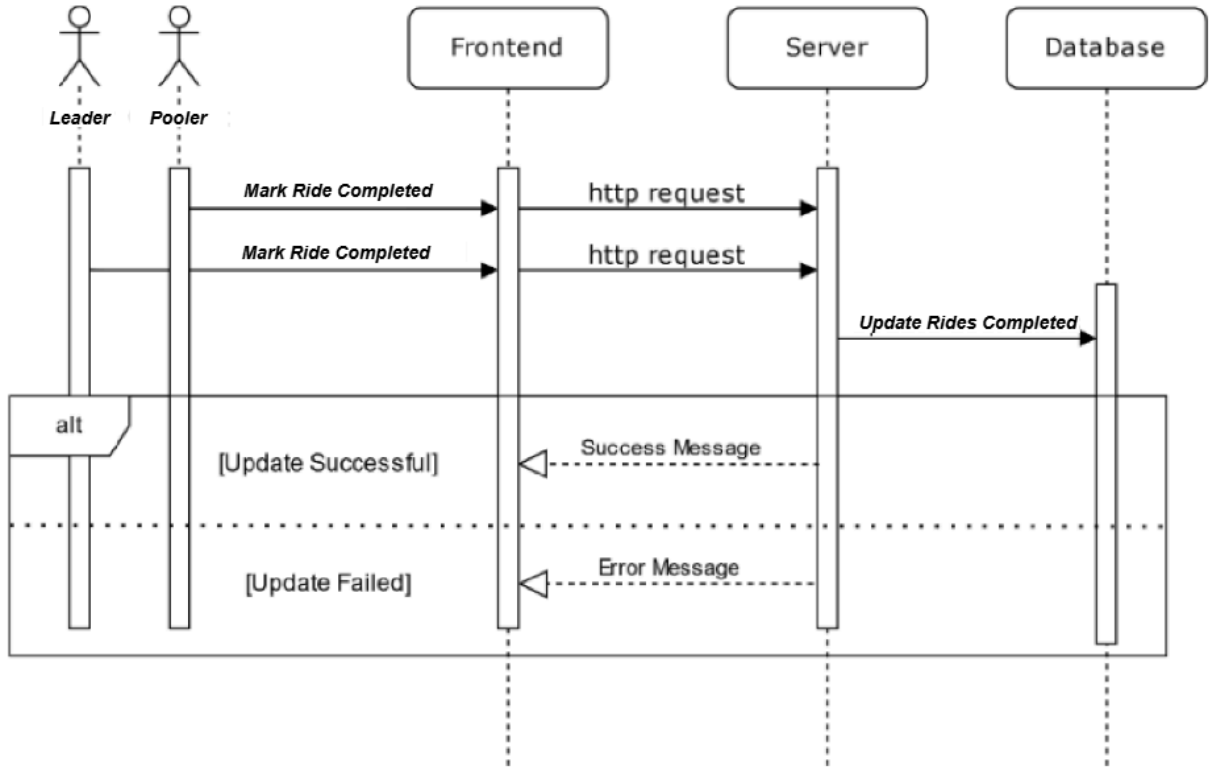
3.3.9 Initiate Chats



Purpose: The Messaging system provides a secure, internal channel for participants to finalize pickup locations and logistical details.

- **Instant Sync:** Utilizes real-time listeners to push messages without delay.
- **Privacy Protection:** Allows coordination without sharing personal phone numbers.
- **Logistical Clarity:** Ensures both parties are aligned on the meeting point

3.3.10 Ride Completed (Trust System)

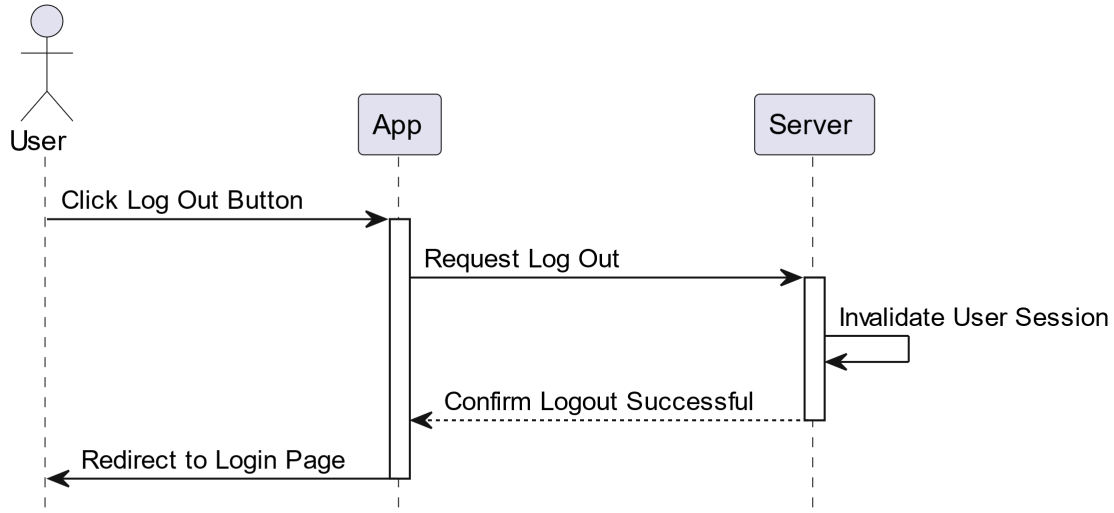


Purpose:

The 'Trust Verification' flow allows users to inspect a Leader's historical performance and reliability metrics before committing to a ride. The Ride is Completed when both Leader and Pooler agree.

- **Social Proof:** Displays the total number of successfully completed rides.
- **Safety Assurance:** Helps users make informed decisions based on peer history.
- **Community Standards:** Encourages accountability among all UniPool participants.

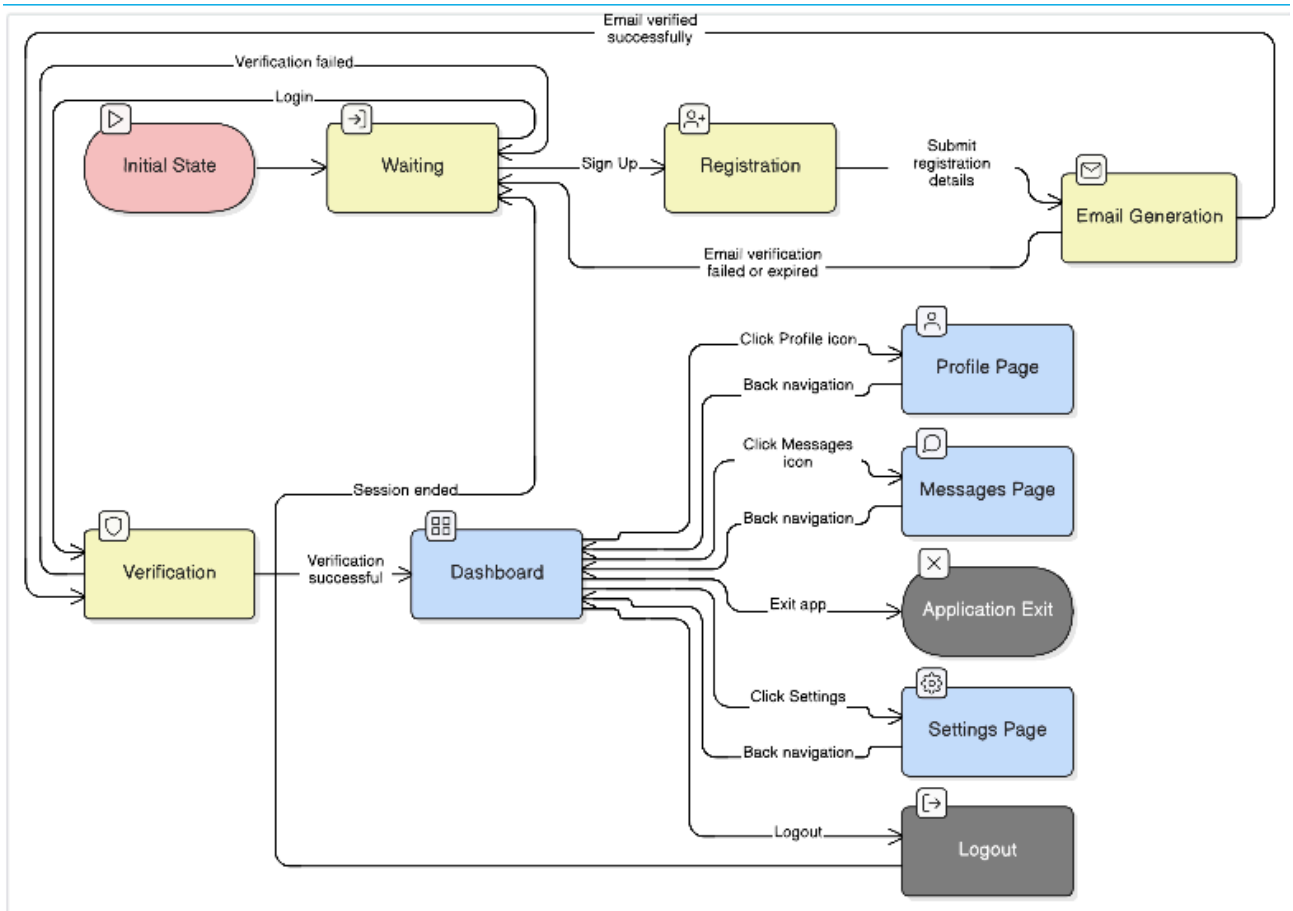
3.3.11 Log out



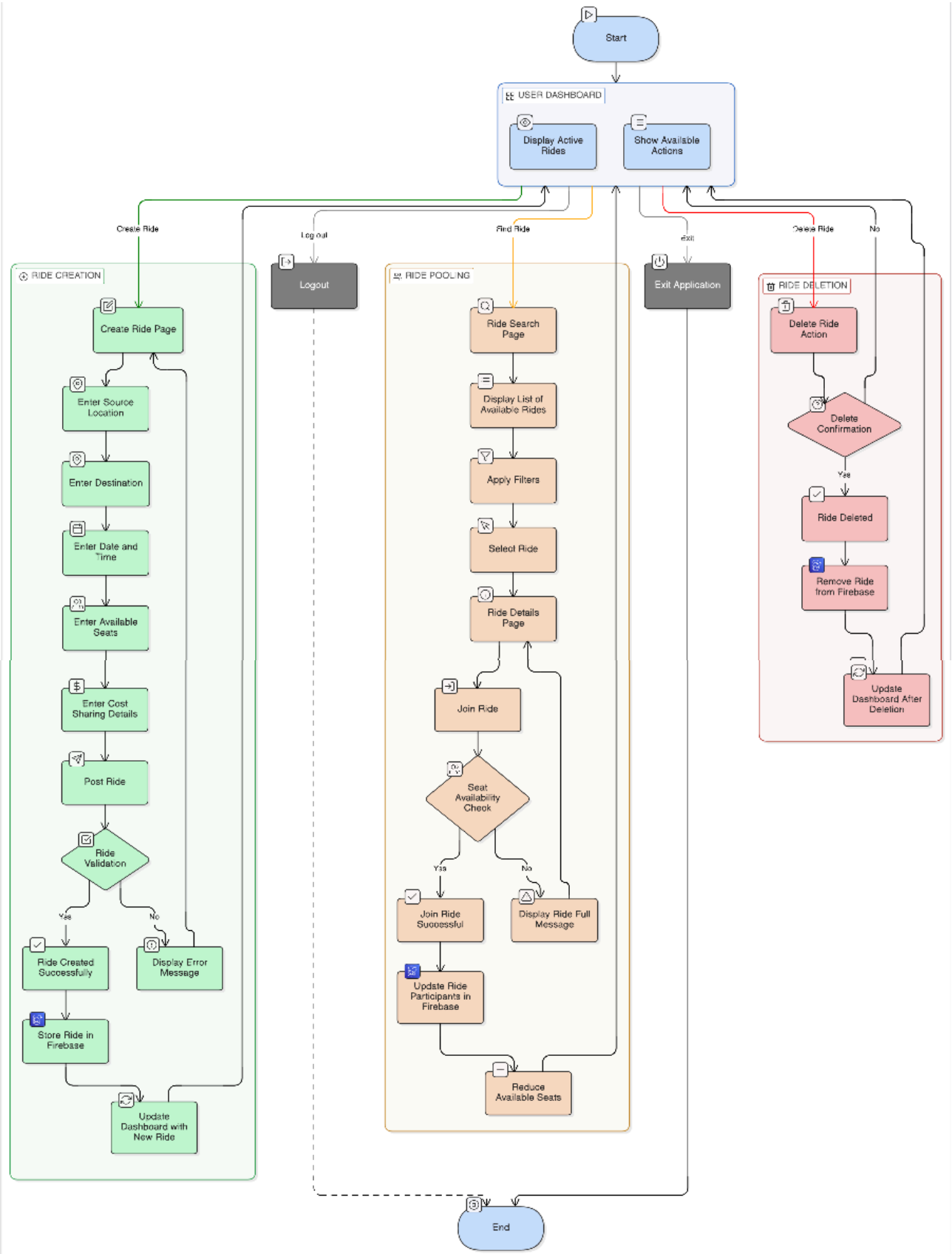
Purpose:
The user can log out of the app on the current device.

3.4 State Diagram

3.4.1 Login and Dashboard



3.4.2 Ride Creation , Browsing and Deletion


Start
USER DASHBOARD
Display Active Rides
Show Available Actions
Create Ride
Log out
Find Ride
Exit
Delete Ride
No
RIDE CREATION
Create Ride Page
Enter Source Location
Enter Destination
Enter Date and Time
Enter Available Seats
Enter Cost Sharing Details
Post Ride
Ride Validation
Yes
No
Ride Created Successfully
Display Error Message
Store Ride in Firebase
Update Dashboard with New Ride
Logout
RIDE POOLING
Ride Search Page
Display List of Available Rides
Apply Filters
Select Ride
Ride Details Page
Join Ride
Seat Availability Check
Yes
No
Join Ride Successful
Display Ride Full Message
Update Ride Participants in Firebase
Reduce Available Seats
Exit Application
RIDE DELETION
Delete Ride Action
Delete Confirmation
Yes
Ride Deleted
Remove Ride from Firebase
Update Dashboard After Deletion
End

The state diagram illustrates the various states and transitions within UniPool. It highlights the interaction flow between different roles, namely **Leader** and **Pooler**, as they navigate through the system.

3.4.1 Overview

The system starts at the application launch, where users authenticate by logging in or registering. After successful authentication, users are directed to the dashboard, which acts as the central navigation state. From the dashboard, users can manage profiles, create or discover rides, view trust information, and communicate through real-time chat. Informational pages are also accessible. Users may log out at any time, returning to the authentication state.

3.4.2 State Descriptions

3.4.2.1 Authentication State

Users must authenticate by logging in or registering to gain access to the system. Successful authentication transitions the user to the dashboard.

3.4.2.2 User State

Authenticated users can access all primary functionalities of the application, including profile management, ride creation and pooling, trust verification, and messaging.

3.4.2.3 Ride Management States

These states allow users to create rides as ride leaders, search and join available rides as poolers, and manage their active rides.

3.4.2.4 Dashboard

The dashboard acts as the central control state of the system. It provides access to informational sections such as About Us, Contact Us, and How to Use, along with editable sections including User Profile, Messages, and Settings for personalization.

4 Project Plan

4.1 Timeline

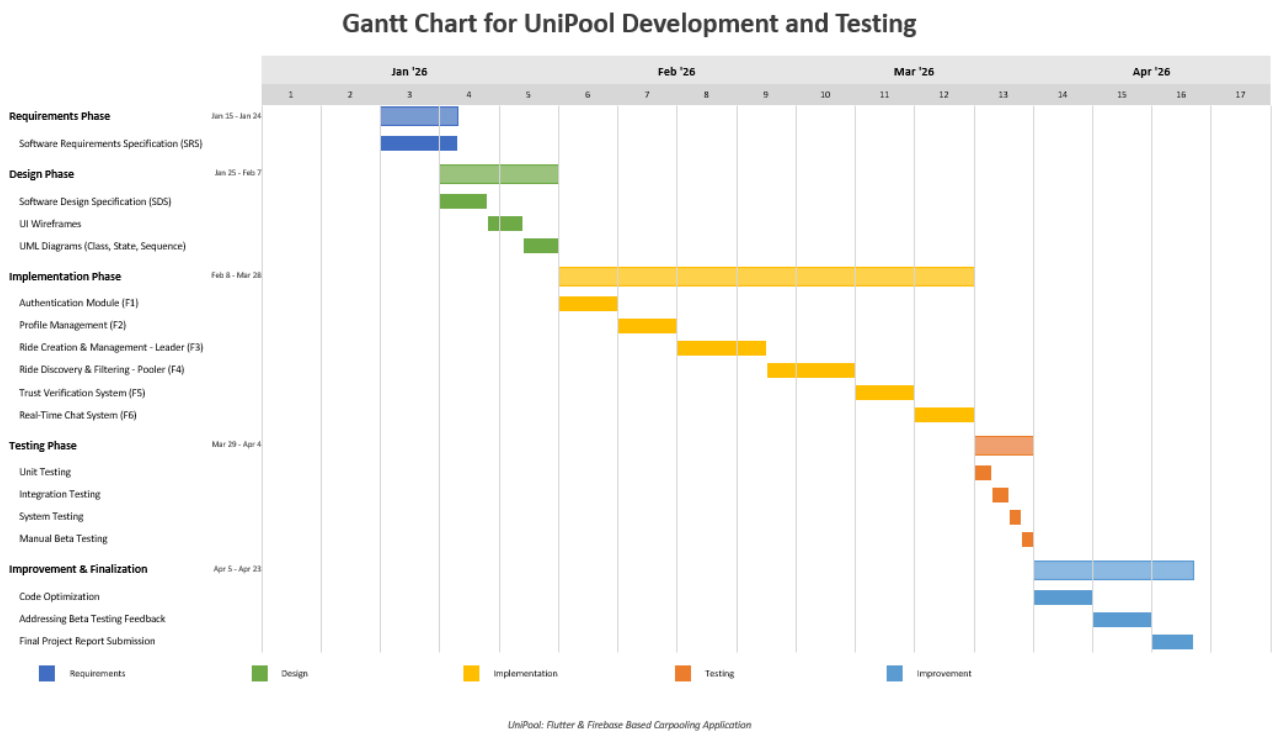


Figure 27: *Gantt chart for development and testing*

15 January 2026 **24 January 2026**	Documentation of Software Requirements Specification (SRS).
25 January 2026 **7 February 2026**	Documentation of Software Design Document (SDS).
8 February 2026 **28 March 2026**	Implementation and testing phase. Development of application features using Flutter and Firebase, including authentication, profile management, ride management, and real-time communication. Execution of unit testing and integration testing by assigned members.
29 March 2026 **4 April 2026**	System-level and manual beta testing were performed to validate functional correctness, usability, and performance. Testing to be conducted from both developer and end-user perspectives to ensure compliance with the specified requirements.
5 April 2026 **13 April 2026**	Code optimization and quality enhancement to be carried out based on beta testing feedback. Improvements focused on performance, security, reliability, and usability, in line with the defined software quality attributes.

14 April 2026 **23 April 2026**	Incorporation of final feedback, verification of all functional and non-functional requirements, and preparation and submission of the final project report.

4.2 Development Phases

This section gives a broad classification of the tasks along with the names of team members responsible for it.

4.2.1 Frontend Design

Frontend development focuses on the design and implementation of all components of the application that directly interact with the end user. This includes translating the user interface requirements defined in the Software Requirements Specification (SRS) into a functional and responsive application interface. All visual elements, user interactions, and navigation flows accessible to the user fall under the scope of frontend development.

The members responsible for frontend development are:

a. Alankrit Gupta
b. Anubhav Rana
c. Aryan Nigam
d. Bhavishya Gupta
e. Naman Mohan Sigh
f. Karan Kumar
g. Gagandeep Kolambe

The following software might be useful for frontend development:

- **Flutter:** Flutter is an open-source UI software development kit developed by Google, enabling developers to build natively compiled applications for mobile, web, and desktop from a single codebase. It features a rich set of customizable widgets for creating expressive UIs, along with a ‘hot reload’ feature for instant UI updates during development. Flutter’s engine, built with C++ and Dart, compiles code to native machine code, ensuring high performance and smooth graphics rendering. This allows for cross-platform development, saving time and effort while maintaining a consistent look and feel across different operating systems.

4.2.2 Backend Development

It is everything that the users don’t see and contains behind-the-scenes activities that occur when performing any action on a website. It focuses primarily on databases, backend logic, APIs, and servers.

The members responsible for backend development are:

a. Aalok
b. Naman Mohan Singh
c. Karan Kumar
d. Pavi Agarwal
e. Harshit Gupta

The following software might be useful for backend development:

- **Firebase:** Firebase is a Backend-as-a-Service (BaaS) platform developed by Google that provides scalable and secure backend infrastructure for application development. In this project, Firebase is used for user authentication, real-time database management, and cloud storage. It enables seamless integration with the Flutter frontend, supports real-time data synchronization, and ensures secure access control through built-in authentication mechanisms. Firebase simplifies backend development by reducing the need for custom server-side code while maintaining reliability, scalability, and data consistency.

Appendix A – Group Log

Date & Time	Venue	Minutes
25/01/26 3:00 pm	1st floor, Rajeev Motwani Building	Initiated user interface design based on the External Interface Requirements. Prepared rough layouts for core screens and assigned Figma wireframe design tasks to team members.
28/01/26 3:00 pm	1st floor, Rajeev Motwani Building	Reviewed UI designs for alignment with defined Leader and Pooler workflows. Discussed Flutter–Firebase code structure and finalized collaboration and repository management practices.
30/02/26 2:00 pm	1st floor, Rajeev Motwani Building	Planned and commenced development of UML diagrams, including state, class, and sequence diagrams, derived from the functional requirements and use cases.
1/02/26 10:00 am	1st floor, Rajeev Motwani Building	Assigned individual responsibilities for diagram preparation. Finalized the development and testing timeline in accordance with project scope.
03/02/26 8:00 pm	1st floor, Rajeev Motwani Building	Reviewed and finalized all designs and diagrams and prepared them for inclusion in the final project documentation. Incorporated changes suggested by our Mentor T.A.